

Axios – End-to-End Technical Implementation Plan

Hohenheim Tomato E-Nose Dataset → ML → Inference → Dashboard

Project: Axios

Data source: University of Hohenheim tomato e-nose datasets

Scope: Static supervised ML + temporal trajectory analysis + inference + dashboard

Current status: Phase 1-4 complete; Phase 5 is next.

This document is the consolidated implementation plan, including the problems discovered during the project, the decisions made because of those problems, and the constraints that must remain fixed through the rest of development.

The original roadmap defines the overall flow from dataset understanding through dashboard/demo.

1. Project Objective

Axios uses the publicly available **University of Hohenheim tomato electronic-nose datasets** to estimate a tomato's position in its storage progression from:

- VOC/e-nose sensor measurements
- weight information
- color measurements
- temporal sensor behavior where available

The system is **not** being presented as a scientifically validated "spoilage detector."

The dataset does not provide a universal biological spoilage endpoint. Therefore, the system will use terminology such as:

- **predicted storage day**
- **storage progression**
- **storage progression stage**
- **remaining observed storage time**

- **sensor degradation trend**

rather than claiming:

"This tomato will spoil in X days."

The original roadmap explicitly establishes this distinction.

2. Scope Freeze

The following is now the fixed scope of Axios.

Included

Dataset

Only:

```
Dataframe_ML_complete.csv  
Dataframe_continuous.csv
```

from the Hohenheim dataset.

ML

- Storage Day regression
- Storage Scenario classification
- temporal trajectory analysis
- derived storage progression
- derived sensor trend
- inference pipeline
- dashboard/demo

Models

Candidate models:

1. Linear Regression
2. Random Forest

3. XGBoost

Optional classification versions:

- Random Forest Classifier
- XGBoost Classifier

The original roadmap intentionally recommends simple models rather than neural networks for this dataset.

Explicitly excluded

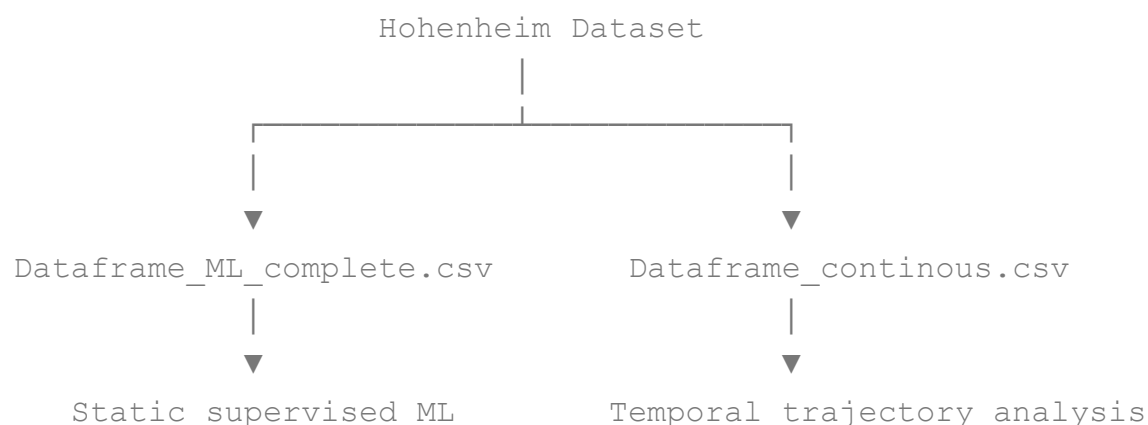
We are **not** doing:

- custom data collection
- SGP41 training
- additional datasets
- synthetic datasets
- deep-learning models
- claiming scientifically validated spoilage prediction
- pretending the Hohenheim continuous and ML datasets are duplicate copies
- combining every available row into one giant training set

This scope constraint is particularly important because the dataset is small. The source roadmap explicitly freezes the project to the Hohenheim data and excludes custom data collection, SGP41 training, and additional datasets.

3. Dataset Architecture

We have two datasets with fundamentally different roles.





The original roadmap makes this distinction explicitly: the ML dataset is primarily for supervised ML, while the continuous dataset is primarily for examining how individual tomatoes change over storage time.

This separation became much more important after Phase 3.

4. Phase 1 – Dataset Inspection

Objective

Before modifying anything, determine exactly what the datasets contain.

The original roadmap requires inspection of:

- number of rows
 - number of columns
 - column names
 - data types
 - missing values
 - duplicate rows
 - unique Sample IDs
 - Storage Scenario
 - Storage Day
 - sensor ranges
 - weight ranges
 - color ranges
 - observations per tomato in the continuous dataset.
-

What we found

Dataframe_ML_complete.csv

```
Rows:      52
Columns:   20
Unique IDs: 18
```

Properties:

- 0 missing values
- 0 duplicate rows
- valid schema
- valid numeric columns
- scenarios:
 - Tc
 - Trt
 - Trtd
- Storage Days:
 - 3
 - 5
 - 7
 - 10
 - 12
 - 14

The day distribution is approximately:

```
Day 3      9
Day 5      9
Day 7      9
Day 10     9
Day 12     7
Day 14     9
```

So even before modeling, the dataset is clearly **small**.

Dataframe_continuous.csv

```
Rows:      90
Columns:   20
Unique IDs: 3
```

It contains:

```
3 Sample IDs
×
3 scenarios
=
9 trajectories
```

Each trajectory contains:

```
10 observations
```

at:

```
3, 4, 5, 6, 7, 10, 11, 12, 13, 14
```

So this dataset is useful for understanding temporal behavior, but it does **not** provide a large independent training population.

Phase 1 output

```
reports/
└─ dataset_profile.json
```

Phase 1 was completed successfully.

5. Problem Discovered: Dataset Is Very Small

This is one of the central constraints of Axios.

We have:

52 ML rows
18 unique Sample IDs

and:

90 continuous rows
3 physical Sample IDs
9 trajectories

A model can easily appear to perform well if we allow information from the same tomato to appear in both training and testing.

Therefore:

Number of rows is not the true effective dataset size.

For leakage purposes, the important unit is the **experimental Sample ID**.

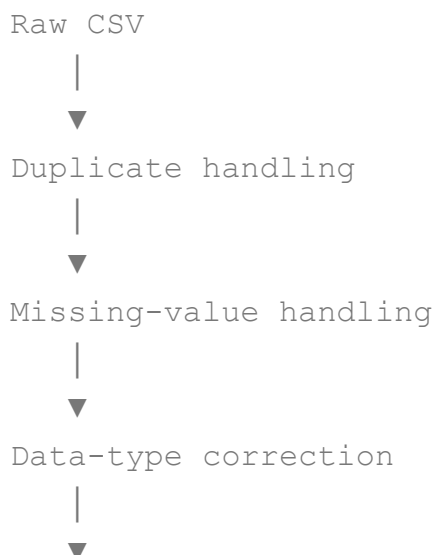
This directly affects Phase 6.

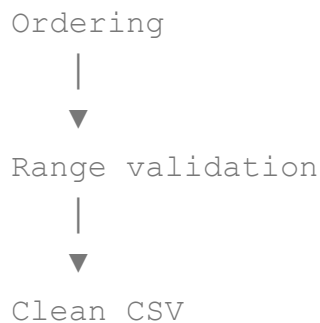
6. Phase 2 – Data Preprocessing

Objective

Convert raw CSVs into validated clean datasets.

The planned pipeline is:





The source roadmap specifically says not to aggressively remove unusual sensor values because unusual VOC readings can represent legitimate observations.

What we actually did

We did **not** blindly:

- remove outliers
- smooth sensor readings
- impute missing data
- normalize everything
- discard unusual VOC measurements

Instead, we validated the data.

Both datasets passed:

- shape preservation
- missing-value validation
- duplicate-row validation
- schema validation
- numeric conversion
- scenario validation
- storage-day validation
- weight validation
- observation-key validation

Outputs

```
data/processed/  
├─ ml_clean.csv  
└─ continuous_clean.csv
```

```
reports/  
└─ phase2_preprocessing_report.json
```

Phase 2 is complete.

7. Phase 3 – Dataset Relationship Analysis

This phase became critical because initially it would have been tempting to think:

```
"We have 52 + 90 = 142 observations."
```

That assumption is incorrect.

We explicitly investigated how the two datasets relate.

Shared observations

There are:

```
9 shared  
(Sample ID, Scenario, Storage Day)
```

keys between the datasets.

However:

```
9 shared keys  
0 exact measurement matches  
9 measurement mismatches
```

Therefore, the datasets should **not** be treated as duplicate representations of the same observations.

8. Problem: ML and Continuous Data Are Not Identical Copies

The shared keys showed differences across measurements.

For example, the diagnostic analysis showed non-zero differences for:

- `minitial`
- `mcurrent`
- `L`
- `a`
- `b`
- `MQ2`
- `MQ3_1`
- `MQ3_10`
- `MQ4`
- `MQ5`
- `MQ6`
- `MQ8`
- `MQ9`
- `MQ135`
- `MQ136`
- `MQ137`
- `MQ138`

Therefore, we made an important architectural decision:

Do not concatenate the 52 ML rows and 90 continuous rows into a 142-row supervised training dataset.

That would incorrectly treat different measurements as though they were duplicate observations of the same experiment.

9. Dataset Role Decision

The final architecture is:

ML dataset

```
52 rows
18 Sample IDs
|
```



Primary supervised ML

Used for:

- Storage Day regression
- Storage Scenario classification
- static features

Continuous dataset

90 rows
9 trajectories
3 Sample IDs
|
▼
Temporal analysis

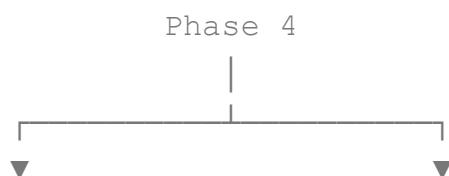
Used for:

- sensor trajectories
- historical feature engineering
- slope
- changes
- rolling statistics
- trend analysis

This preserves the information contained in both datasets without artificially increasing the effective training population.

10. Phase 4 – Feature Engineering

Phase 4 was divided into two branches because of the dataset-role decision.



The roadmap specifies these two feature groups separately.

11. Phase 4A – Static Features

Input:

```
data/processed/ml_clean.csv
```

Output:

```
data/features/static_features.csv
```

Raw static features

Gas sensors

```
MQ2  
MQ3_1  
MQ3_10  
MQ4  
MQ5  
MQ6  
MQ8  
MQ9  
MQ135  
MQ136  
MQ137  
MQ138
```

Physical/color measurements

```
minitial
mcurrent
L
a
b
```

These correspond to the features specified in the roadmap.

Derived feature

We calculate:

```
$$
weight_loss_pct =
\frac{m_{\text{initial}} - m_{\text{current}}}{m_{\text{initial}}}
\times 100
$$
```

This gives the relative weight loss.

Observed:

```
Minimum: 0.8312%
Maximum: 9.9390%
Mean:    4.2990%
Std:     2.1629%
```

Important target leakage rule

`Storage Day` exists in the feature table because it is part of the dataset identity/target information.

But:

Storage Day must never be supplied to a model whose job is to predict Storage Day.

The same applies to `Storage Scenario` when training the scenario classifier.

The feature table is therefore an intermediate analytical artifact, not automatically the final model matrix.

12. Phase 4B – Temporal Features

Input:

```
data/processed/continuous_clean.csv
```

Output:

```
data/features/temporal_features.csv
```

We have:

```
90 rows
9 trajectories
88 columns
```

For each:

```
Sample ID + Storage Scenario
```

we sort chronologically by Storage Day.

For every gas sensor

We calculate:

1. Current

Current sensor reading.

```
MQ2_current
```

2. Change

Difference from the previous observation.

MQ2_change

3. Slope

Least-squares slope using all observations available up to the current point.

This is important because the feature must remain causal.

We cannot use future observations to calculate a feature for an earlier observation.

4. Rolling mean

Window:

```
3 observations
```

including the current observation.

5. Rolling standard deviation

Again using the three-observation causal window.

6. Rolling minimum

7. Rolling maximum

First-observation behavior

At the beginning of each trajectory there is no historical observation.

Therefore:

```
change = 0
slope = 0
rolling_std = 0
rolling_min = current
rolling_max = current
```

and:

```
history_count = 1
```

This avoids inventing nonexistent history.

Temporal validation

Phase 4B verified:

```
Missing values:      0
Duplicate rows:      0
Duplicate full keys:  0
Trajectory lengths:  correct
Chronological order: correct
Causal violations:    0
```

Phase 4 is therefore complete.

13. Major Problem: Temporal Dataset Is Too Small for General Training

The continuous dataset has:

```
3 Sample IDs
9 trajectories
```

That is nowhere near enough independent experimental units to claim that a temporal model generalizes broadly.

Therefore:

Temporal features are not automatically merged into the 52-row static ML training dataset.

They will instead be used where the data actually supports them:

- trajectory analysis

- trend visualization
- derived trend outputs
- potentially a carefully controlled temporal experiment if justified later

This is a deliberate limitation, not an implementation failure.

14. Phase 5 – Target Construction

This is our next phase.

The roadmap originally defines three target concepts.

We need to formalize them carefully because the actual dataset structure exposed an issue with Target 3.

15. Target 1 – Storage Day

This is the primary target.

X:
VOC + weight + color features

↓

Regression model

↓

predicted Storage Day

Example:

Actual Storage Day:	10
Predicted Storage Day:	8.7

Evaluation:

MAE
RMSE
 R^2

Why this is the primary target

It is directly present in the Hohenheim dataset.

We do not need to invent a biological threshold.

We are asking:

Can the measured e-nose/physical features identify the tomato's position in the observed storage timeline?

That is a dataset-supported question.

16. Target 2 – Storage Scenario

The dataset contains:

Tc
Trt
Trtd

Therefore we can construct a classification task:

```
Features
  |
  ▼
Classifier
  |
  ▼
Tc / Trt / Trtd
```

This is explicitly defined as a secondary experiment in the roadmap.

Metrics:

Accuracy
Precision
Recall
F1
Confusion Matrix

However, because the dataset is small, we will treat this as a secondary result rather than allowing it to dominate the project.

17. Target 3 – Remaining Observed Storage Time

The original roadmap proposes:

```
$$  
remaining_storage =  
maximum\ observed\ day-current\ day  
$$
```

and explicitly distinguishes this from scientifically validated shelf life.

However, our actual dataset structure requires an important correction.

Problem

The ML dataset has essentially one observation per:

```
Sample ID + Scenario
```

Therefore, for a static ML row, we do not have a trajectory showing:

```
Day 3  
Day 5  
Day 7  
Day 10  
...
```

for the same experimental unit.

If we try to derive "remaining time" from that single ML observation, we risk constructing a target that is simply a mathematical transformation of Storage Day rather than an independently learned biological/temporal quantity.

The continuous dataset does contain trajectories, but only:

```
3 Sample IDs
9 trajectories
```

So it is too small to support a general-purpose remaining-time model.

Decision

We will **not blindly train a separate remaining-storage regressor**.

Instead:

Primary model

Predict:

```
predicted_storage_day
```

Derived output

Given the observed experimental window:

```
maximum observed day = 14
```

we can derive:

\$\$

remaining\ observed\ window

14 - predicted\ storage\ day
\$\$

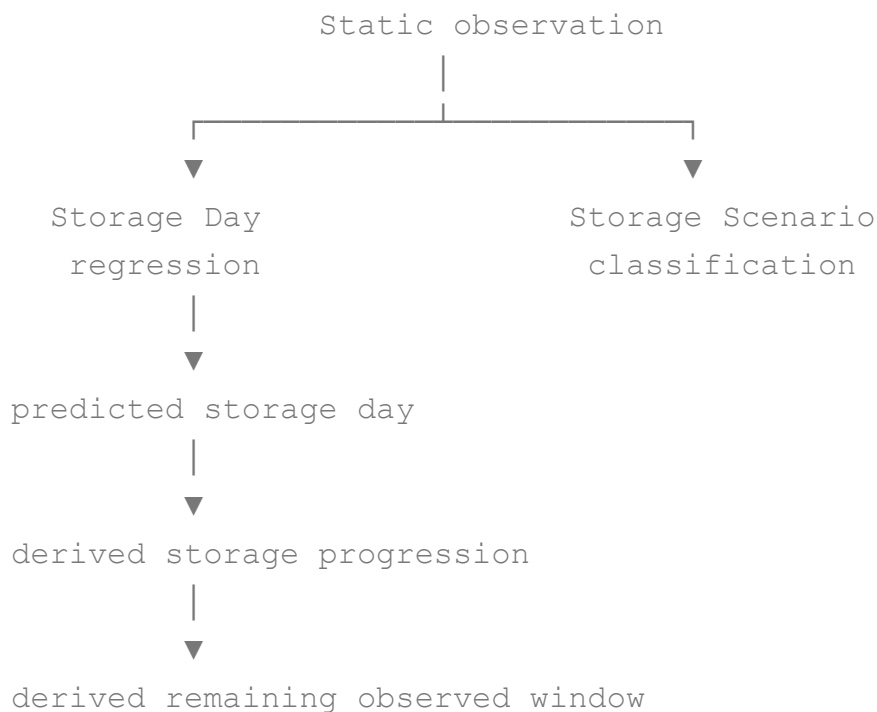
when appropriate.

This is a **derived observed-window quantity**, not an independently validated shelf-life prediction.

That is methodologically cleaner.

18. Phase 5 Target Contract

The target architecture should therefore become:



Not:

three independent ML models

unless later evidence from the data justifies the third model.

19. Phase 6 – Leakage-Safe Train/Test Split

This phase is extremely important.

The dataset contains repeated observations associated with experimental units.

Therefore:

We split by `Sample ID`, not by individual rows.

The roadmap explicitly identifies this as a leakage concern.

Wrong

```
Tomato A Day 3 → TRAIN
```

```
Tomato A Day 7 → TEST
```

The model has effectively seen the same tomato during training.

That produces an overly optimistic evaluation.

Correct

```
TRAIN
```

```
Sample IDs:
```

```
A
```

```
B
```

```
C
```

```
D
```

```
...
```

```
TEST
```

```
Sample IDs:
```

```
X
```

```
Y
```

```
Z
```

Every observation associated with a particular experimental unit stays in one partition.

20. Small-Dataset Problem in Phase 6

We only have:

```
18 unique Sample IDs
```

So a conventional:

```
80/20 random row split
```

is inappropriate.

Even an 80/20 **group split** gives only a few independent test groups.

Therefore, we should use group-aware evaluation rather than relying on one arbitrary split.

The exact implementation can use:

- grouped holdout
- GroupKFold
- repeated grouped evaluation where feasible

while preserving the strict rule that a Sample ID cannot cross train/test.

The evaluation report must explicitly state the number of independent test groups.

21. Phase 7 – Model Training

We deliberately keep the model family small.

The roadmap specifies three regression candidates.

Model A – Linear Regression

Purpose:

Establish a simple baseline.

Pipeline:

```
Features
```

↓

Linear Regression



Storage Day

This tells us whether the relationship is approximately linear.

22. Model B – Random Forest

Purpose:

Establish a nonlinear tree-based baseline.

Pipeline:

Features



Random Forest Regressor



Storage Day

Advantages for this dataset:

- nonlinear relationships
- feature interactions
- little preprocessing
- useful feature importance diagnostics

But we will not assume it is better before evaluation.

23. Model C – XGBoost

Purpose:

Strong candidate for the primary regression model.

Pipeline:

Features



XGBoost Regressor



Storage Day

Again:

XGBoost is a candidate, not the predetermined winner.

The roadmap originally describes XGBoost as the primary candidate, but final model selection must be based on the actual leakage-safe evaluation results.

24. Classification Models

For:

Storage Scenario

we can evaluate:

Random Forest Classifier

XGBoost Classifier

Potentially with a simple baseline depending on class distribution.

We do not need neural networks.

25. Why No Neural Network?

This is an intentional decision based on the data scale.

The static dataset contains:

52 rows

18 independent Sample IDs

A neural network would have a very high capacity relative to the available independent observations.

That makes overfitting and unstable evaluation particularly serious.

A tree-based model or simple regression model is much easier to:

- train
- validate
- interpret
- debug
- compare

The original roadmap likewise explicitly says that neural networks are unnecessary for this stage.

26. Phase 8 – Model Evaluation

Regression metrics

For Storage Day:

MAE

Mean absolute error.

Example interpretation:

$$\text{MAE} = 1.2 \text{ days}$$

means predictions are off by approximately 1.2 days on average in absolute terms.

RMSE

Penalizes larger errors more strongly.

R^2

Measures explained variance.

But because the dataset is small, R^2 must be interpreted carefully.

A high R^2 from a tiny test population does not automatically demonstrate strong generalization.

27. Classification Metrics

For Storage Scenario:

```
Accuracy
Precision
Recall
F1
Confusion Matrix
```

The confusion matrix is particularly useful because it tells us which scenarios are being confused with each other.

28. Important Evaluation Rule

We will not simply produce:

```
XGBoost = 0.95
```

and declare success.

The evaluation report should include:

- number of independent Sample IDs
- split strategy
- training groups
- test groups
- target distribution
- model
- features
- metrics
- predictions
- residual/error analysis

This is necessary because the effective dataset is small.

29. Phase 9 – Final Model Selection

Only after evaluation do we select the models.

Possible artifact structure:

```
artifacts/  
└─ models/  
    └─ storage_day_model.pkl  
    └─ scenario_model.pkl
```

If an independently justified remaining-time model is eventually supported, then:

```
remaining_storage_model.pkl
```

can be added.

But we should not create an artifact merely because the original roadmap listed one.

The source roadmap itself says the exact final model depends on evaluation results.

30. Model Metadata

Every saved model should ideally have associated metadata:

```
model name  
model version  
training dataset  
feature list  
target  
split strategy  
training Sample IDs  
validation/test Sample IDs  
hyperparameters  
metrics  
timestamp
```

This prevents the dashboard from accidentally loading a model trained with a different feature contract.

31. Phase 10 – Inference Pipeline

This is where the project transitions from:

"We trained a model"

to:

"We have an actual system."

The roadmap defines `inference.py` as the central inference component.

Input

A new Hohenheim-format observation.

Conceptually:

```
{  
    "MQ2": ...,  
    "MQ3_1": ...,  
    ...  
    "MQ138": ...,  
    "minitial": ...,  
    "mcurrent": ...,  
    "L": ...,  
    "a": ...,  
    "b": ...  
}
```

Pipeline

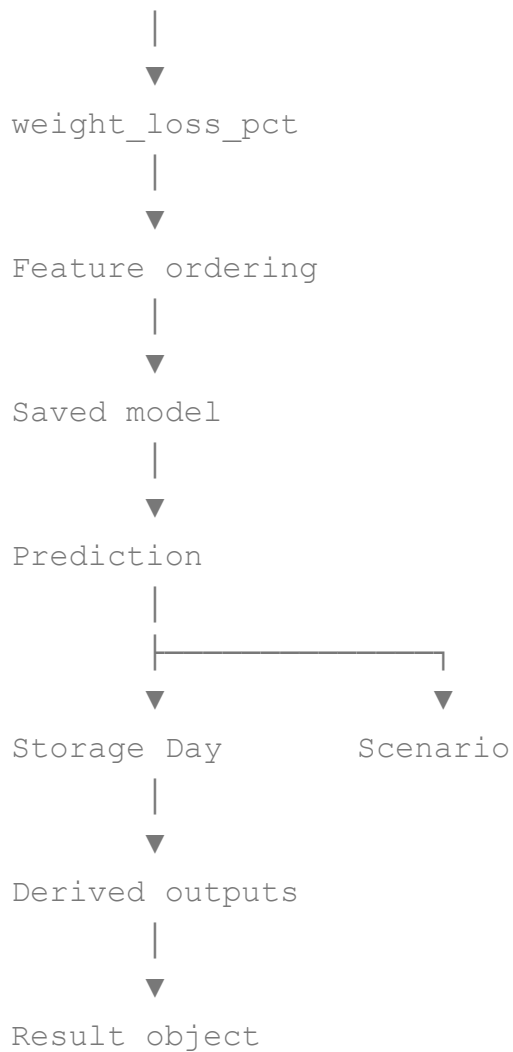
New observation



Input validation



Feature construction



32. Inference Contract

Something conceptually like:

```
{
  "predicted_storage_day": 8.4,
  "storage_progression": "LATE",
  "estimated_remaining_observed_days": 5.6,
  "storage_scenario": "Trt"
}
```

But the actual output will be carefully named so it does not imply validated shelf life.

33. Phase 11 – Derived Outputs

This phase converts raw predictions into interpretable information.

The roadmap defines four storage progression states.

Storage progression

0-3	EARLY
4-7	MID
8-10	LATE
11-14	VERY_LATE

These are **storage progression stages**.

They are not:

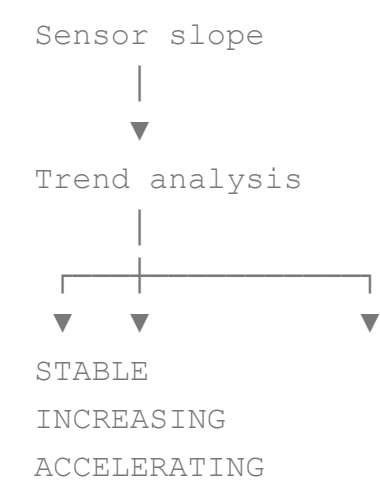
- FRESH
- GOOD
- BAD
- SPOILED

because the source dataset does not provide a defensible universal spoilage threshold.

34. Sensor Trend

The continuous dataset can also provide a trajectory-based interpretation.

Conceptually:



The exact trend logic should be defined explicitly rather than manually eyeballed.

For example, it may use:

- slope
- recent slope
- slope change
- multiple sensor agreement

rather than one arbitrary sensor.

35. Remaining Observed Time

The dashboard may show:

```
Remaining observed window:  
5.6 days
```

but the UI must communicate what this means.

Preferred terminology:

Estimated remaining observed storage window

Not:

Shelf life remaining

and definitely not:

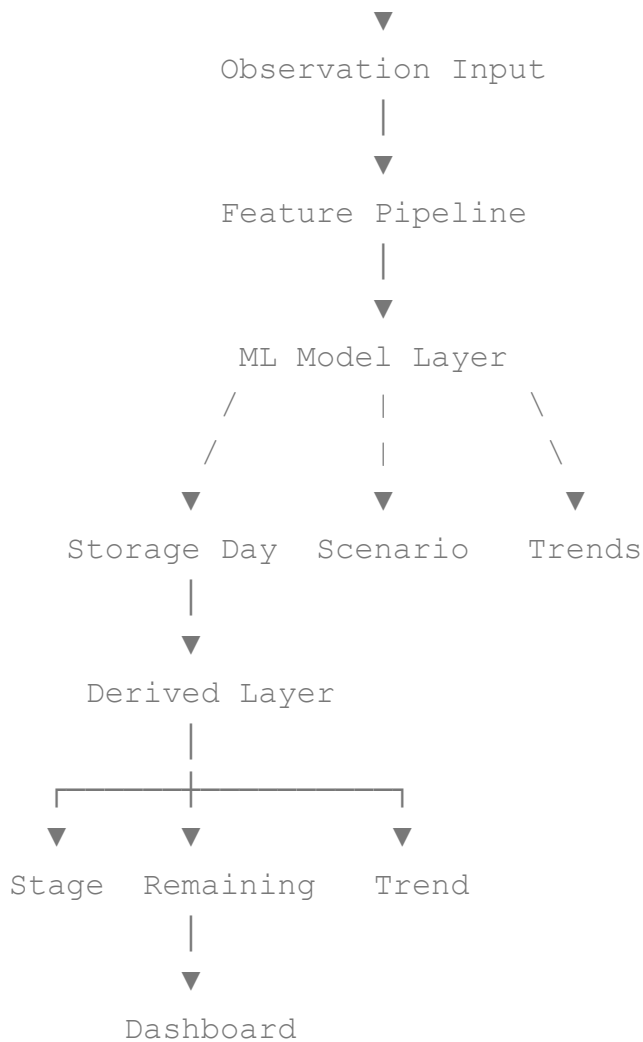
Time until spoilage

unless future evidence establishes that relationship.

36. Phase 12 – Dashboard / Demo

The final user-facing architecture becomes:

Axios
|



The original roadmap describes the final application in essentially this structure.

37. Dashboard – Main Information

The dashboard should display something approximately like:

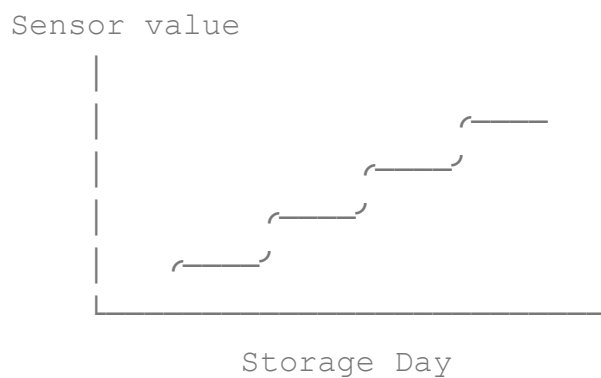
AXIOS	
Predicted Storage Day	8.4
Storage Progression	LATE
Estimated Remaining Observed Window	5.6 days

Storage Scenario
Trt
Sensor Trend
INCREASING

The dashboard should also show the underlying trajectory rather than only a single prediction.

38. Dashboard – Sensor Trajectory

For the continuous data:



Potential views:

- individual sensor
- grouped VOC trends
- rolling average
- slope
- weight loss
- color progression

The purpose is to make the model output traceable to the underlying measurements.

39. Dashboard – Explainability

Because the dataset is small, the dashboard should avoid pretending the prediction is more certain than it is.

Useful additions:

Prediction
Model
Key contributing features
Observed range
Data coverage

Potential feature importance:

Feature	Importance

MQxxx	...
weight_loss_pct	...
MQxxx	...
L	...
...	

For tree models, feature importance can be displayed, with the exact interpretation depending on the importance method used.

40. Dashboard – Scientific Disclaimer

The dashboard should contain a concise statement such as:

Axios estimates storage progression from the Hohenheim tomato e-nose dataset. "Remaining observed storage" refers to the observed experimental storage window and is not a validated biological shelf-life prediction.

This protects the project from making a stronger scientific claim than the data supports.

41. Final Repository Architecture

The project should converge toward:

Axios/

```
|
|
|— data/
|   |— raw/
|   |   |— Dataframe_ML_complete.csv
|   |   |— Dataframe_continuous.csv
|   |
|   |— processed/
|   |   |— ml_clean.csv
|   |   |— continuous_clean.csv
|   |
|   |— features/
|   |   |— static_features.csv
|   |   |— temporal_features.csv
|   |
|   |— targets/
|   |   |— supervised_targets.csv
|   |
|   |— splits/
|   |   |— train.csv
|   |   |— test.csv
|   |
|— artifacts/
|   |— models/
|   |   |— storage_day_model.pkl
|   |   |— scenario_model.pkl
|   |
|   |— evaluation/
|   |   |— regression_results.json
|   |   |— classification_results.json
|   |   |— predictions.csv
|   |
|— scripts/
|   |— phase1/
|   |   |— inspect_dataset.py
|   |
|   |— phase2/
|   |   |— preprocess_dataset.py
|   |
|   |— phase3/
|   |   |— analyze_dataset_relationship.py
|   |
|   |— phase4/
|   |   |— build_static_features.py
```

```
| | | └─ build_temporal_features.py
| | |
| | └─ phase5/
| | └─ phase6/
| | └─ phase7/
| | └─ phase8/
| | └─ phase9/
| | └─ phase10/
| | └─ phase11/
|
└─ src/
    └─ data/
    └─ features/
    └─ models/
    └─ evaluation/
    └─ inference/

└─ reports/
    └─ dataset_profile.json
    └─ phase2_preprocessing_report.json
    └─ phase3_relationship_report.json
    └─ phase4_static_features_report.json
    └─ phase4_temporal_features_report.json

└─ dashboard/

└─ tests/

└─ notebooks/

└─ requirements.txt
└─ README.md
```

42. Complete Phase-by-Phase Plan

Phase	Work	Main output
1	Dataset inspection	Dataset profile
2	Cleaning + validation	Clean CSVs

Phase	Work	Main output
3	Dataset relationship analysis	Validated dataset roles
4	Static + temporal feature engineering	Feature datasets
5	Target construction	Target contract/dataset
6	Leakage-safe grouped split	Train/test partitions
7	Model training	Candidate models
8	Model evaluation	Metrics + predictions
9	Final model selection	Saved model artifacts
10	Inference pipeline	<code>inference.py</code>
11	Derived outputs	Stage/trend/remaining-window logic
12	Dashboard/demo	Working Axios application

This is the frozen implementation structure derived from the roadmap, with the actual dataset discoveries incorporated into the design.

43. The Important Problems We Have Already Solved

Problem 1 – Very few observations

```
52 ML rows
18 Sample IDs
```

Decision

Use simple models and strict group-aware evaluation.

Problem 2 – Continuous dataset is tiny

```
90 rows
3 Sample IDs
9 trajectories
```

Decision

Use it primarily for temporal analysis, not as an independent general-purpose training population.

Problem 3 – Two datasets are not duplicate copies

9 shared observation keys had:

```
0 exact matches
9 mismatches
```

Decision

Do not concatenate the datasets.

Problem 4 – Potential target leakage

`Storage Day` exists directly in the source data.

Decision

Keep it for target construction but never feed it into a Storage Day prediction model.

Problem 5 – Temporal leakage

Historical/future measurements can accidentally leak across training and test sets.

Decision

Temporal features are causal/past-only.

Problem 6 – Sample leakage

Rows belonging to the same tomato can otherwise appear in train and test.

Decision

Split by `Sample ID`.

Problem 7 – No true spoilage label

The dataset does not establish:

```
spoiled = x
```

at a scientifically universal point.

Decision

Predict storage progression rather than claim spoilage detection.

Problem 8 – Remaining storage target is questionable

The continuous data supports trajectories, but only for 3 Sample IDs.

The static dataset doesn't provide a complete trajectory for each ML row.

Decision

Do not blindly train a third regressor.

Use remaining observed window primarily as a derived quantity unless later analysis establishes a defensible independent target.

Problem 9 – Overfitting risk

A 52-row dataset cannot justify a complex neural architecture.

Decision

Use:

```
Linear Regression
Random Forest
XGBoost
```

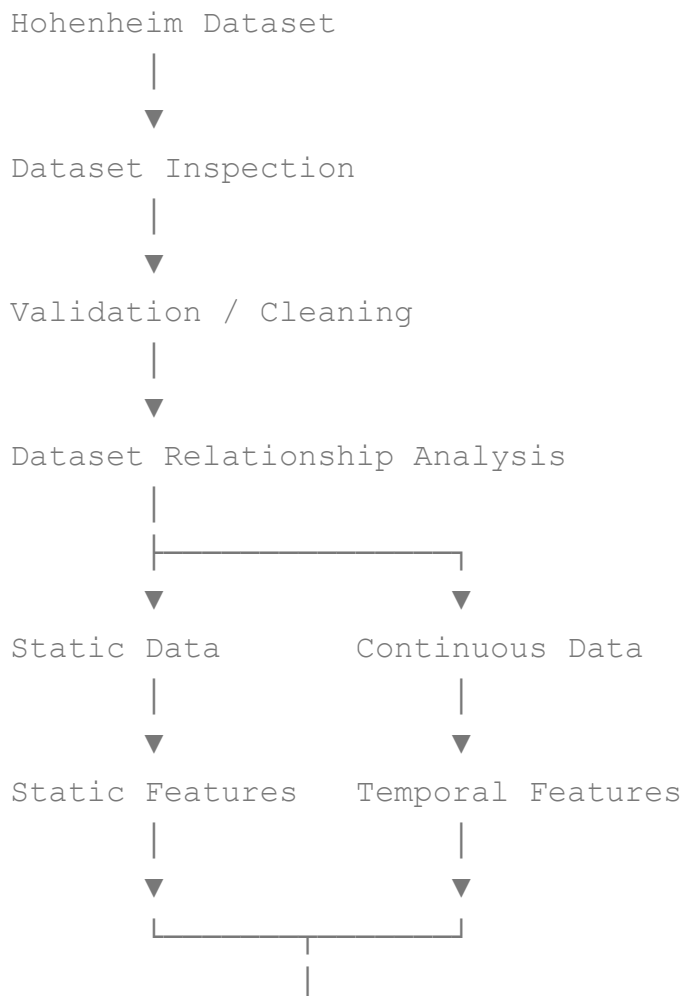
and evaluate honestly.

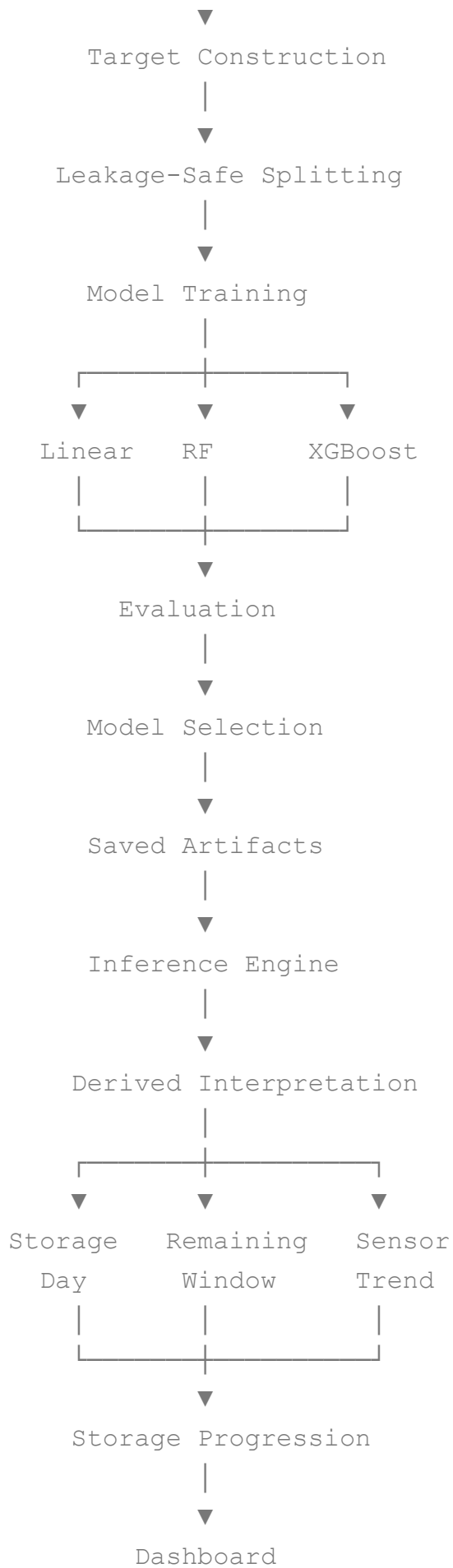
44. What Axios Ultimately Demonstrates

The final system is not simply:

```
CSV → XGBoost → prediction
```

It demonstrates a complete ML engineering pipeline:





The key engineering principle throughout is:

Every downstream claim must be constrained by what the Hohenheim data can actually support.

That is why we have deliberately accepted the small dataset, separated the two dataset roles, avoided artificial data expansion, prevented leakage, avoided unsupported spoilage labels, and kept temporal information separate from the main static training population.

45. Current State

Completed:

Phase 1 ✓
Phase 2 ✓
Phase 3 ✓
Phase 4A ✓
Phase 4B ✓

Current:

Phase 5 → Target Construction

Next implementation should therefore be:

`scripts/phase5/`

with a formal target-generation script and validation report.

The first thing we should implement in Phase 5 is **the target contract**, specifically resolving `Storage Day`, `Storage Scenario`, and the derived remaining-observed-window quantity without creating a misleading third ML target.